



ABC ReKYC System

Technical & Design document

Revision History

Version	Change Description	Date	Prepared By	Reviewed By	Approved By
1.0	Initial customer level rekyc	12.03.2026	Tamizhan M	Jeevith Priya	
2.0	Updated flow with UCIC level and multiple Re-KYC records for a single UCIC based on LAN	03.04.2026	Tamizhan M	Jeevith Priya	

Table of Contents

1. INTRODUCTION.....	5
2. PURPOSE	6
3. SCOPE	7
4. ABBREVIATIONS / DEFINITIONS	7
5. DESIGN TOOLS.....	8
6. SYSTEM OVERVIEW.....	9
6.1 BUSINESS ARCHITECTURE:	9
6.1.1 AUTHENTICATION & SECURITY MODULE.....	9
6.1.2 CUSTOMER PORTAL MODULE	10
6.1.3 REMINDER ENGINE MODULE.....	10
6.1.4 SFDC CASE MANAGEMENT MODULE	11
6.1.5 ADMIN DASHBOARD MODULE	12
6.1.6 DWH SYNCHRONIZATION MODULE.....	12
6.1.7 AUDIT & HISTORY MODULE	13
6.2 APPLICATION ARCHITECTURE.....	13
6.2.1 WEB CLIENT (REACT).....	13
6.2.1.1 LIBRARY VERSIONS.....	13
6.2.1.2 KEY PACKAGES.....	14
6.2.1.3 KEY RE-USABLE COMPONENTS	15
6.2.2 BACKEND APPLICATION (SPRING BOOT)	18
6.2.2.1 TECHNOLOGY STACK	18
6.2.2.2 BEST PRACTICES FOLLOWED	19
6.2.2.3 KEY PACKAGES.....	19
6.2.2.4 API ENDPOINTS SUMMARY.....	20
6.2.3 SECURITY ARCHITECTURE	22
6.2.3.1 CUSTOMER AUTHENTICATION (OTP TO JWT)	22
6.2.3.2 ADMIN AUTHENTICATION (CYBERARK OIDC).....	22
6.2.3.3 JWT FILTER CHAIN	22
6.2.4 THREE-PHASE CUSTOMER UPDATE ARCHITECTURE	23
6.2.5 SCHEDULER ARCHITECTURE	24
6.2.6 DATABASE LAYER (MYSQL)	25
6.2.7 DOCUMENT STORAGE (AWS S3)	26
6.2.8 EXTERNAL SYSTEM INTEGRATIONS	27
6.2.9 IN-APP NOTIFICATION DESIGN.....	28
6.3 DEPLOYMENT ARCHITECTURE	29
6.3.1 CI/CD PIPELINE WITH JENKINS	29
6.3.2 TRAFFIC FLOW & LOAD BALANCING.....	31
6.3.3 APPLICATION CONTAINER	31
6.3.4 DATABASE & STORAGE LAYER.....	31
6.3.5 MONITORING & LOGGING	32
6.3.6 KEY ENVIRONMENT CONFIGURATION	32
7. GOALS AND CONSTRAINTS	33
7.1 USABILITY	33
7.2 SECURITY.....	33

7.3 PLATFORM DEPENDENT	34
7.4 PERFORMANCE	34
7.4.1 TRANSACTIONS	34
7.4.2 CONCURRENCY SUPPORT	34
7.5 MAINTAINABILITY	35
7.5.1 COMPONENTS AND OBJECTS	35
7.5.2 THIRD PARTY VENDORS	35
7.6 SCALABILITY	36
7.7 RELIABILITY	36
7.8 COMPATIBILITY	37
7.8.1 DATABASE.....	37
7.8.2 TECHNOLOGY	37
8. DESIGN.....	38
8.1 THE USER INTERFACE ENVIRONMENT.....	38
8.2 MODULE DETAILS.....	38
8.2.1 MASTER DATA.....	38
8.2.2 DATABASE.....	39
8.2.3 INTEGRATION SEQUENCE	39
8.2.4 REPORTS.....	42

1. Introduction

The ABC ReKYC (Re-Know Your Customer) System is a digital platform designed, developed, and implemented to automate the periodic re-verification of customer KYC (Know Your Customer) information as mandated by regulatory requirements.

Built on the Spring Boot 3.3.x framework, the ReKYC system follows a monolithic layered architecture deployed in a containerized environment. The source code is managed using Git and stored in a private Bitbucket repository. The system is deployed using a Docker-based model, ensuring scalability, flexibility, and efficient resource utilization.

The application is a REST API backend that powers both a customer-facing web portal and an administrative dashboard. It integrates with multiple external systems including Salesforce (SFDC) for case management, Databricks (DWH) for customer data synchronization, CyberArk for admin identity management, AWS S3 for document storage, and third-party Email/SMS services for customer notifications.

The system automates the complete ReKYC lifecycle:

- ◆ Identification of customers due for re-verification based on risk classification
- ◆ Automated multi-stage reminder campaigns (Email + SMS)
- ◆ Secure customer portal access via OTP authentication
- ◆ Digital submission of updated KYC information and supporting documents
- ◆ Automated Salesforce case creation and tracking for changes requiring salesforce review
- ◆ Admin dashboard for real-time compliance monitoring and reporting
- ◆ Bidirectional data synchronization with the central Data Warehouse

In summary, the system design includes the following sub-systems:

- ◆ ReKYC Backend REST API Application
- ◆ Customer Portal (Frontend Web Application)
- ◆ Admin Dashboard (Frontend Web Application)
- ◆ MySQL Database
- ◆ AWS S3 Document Storage
- ◆ Scheduler Jobs (Reminder, SFDC Query, DWH Sync)
- ◆ External System Integrations (Salesforce, CyberArk, DWH, Email/SMS, PAN API)

- ◆ GIT Version Control — Bitbucket Source Code Control

Dual-Mode Re-KYC Tracking:

The system now supports two Re-KYC tracking modes, configurable via `app.rekyc.level`.

Mode	Tracking	Tracking Entity	Use Case
UCIC	Per person	`customer_ucics`	One Re-KYC covers all loans of a person
LAN	Per loan	`loan_accounts`	Each loan account has its own Re-KYC cycle

2. Purpose

The purpose of this document is to provide a High Level Design Architecture for the ABC ReKYC System. The Design document is intended to capture and convey the significant architectural decisions which have been made on the system, including technology stack, system components, integration points, data flows, and deployment strategy.

This document serves as the primary reference for:

- ◆ Development and QA teams for implementation guidance
- ◆ Business stakeholders for architectural review and approval
- ◆ Operations teams for deployment and infrastructure planning
- ◆ Compliance teams for audit and regulatory verification

3. Scope

The ReKYC system automates the mandatory periodic re-verification of customer KYC information. The main objectives are to ensure regulatory compliance, reduce operational costs, provide real-time compliance visibility, and maintain a complete audit trail of all customer changes.

Main modules are:

- ◆ Authentication & Security — OTP-based customer login, CyberArk SSO for admins, JWT-based access control
- ◆ Customer Portal — Customer profile view, information update, beneficiary management, document upload
- ◆ Reminder Engine — 6-stage automated reminder campaign (Email + SMS) with TinyURL links
- ◆ SFDC Case Management — Automated Salesforce case creation for customers who submit changes
- ◆ Admin Dashboard — Real-time compliance metrics, customer tracking, risk analysis, reports
- ◆ DWH Integration — Bidirectional sync with Databricks Data Warehouse
- ◆ Audit & History — Complete field-level change history and scheduler execution logs

4. Abbreviations / Definitions

Term	Explanation
API	Application Programming Interface
AWS S3	Amazon Web Services Simple Storage Service
CyberArk	Corporate Identity and Privileged Access Management system
DWH	Data Warehouse (Databricks)
HLD	High Level Design
JWT	JSON Web Token — secure token for authenticated API access
KYC	Know Your Customer — regulatory identity verification process
OIDC	OpenID Connect — identity layer on top of OAuth 2.0
OTP	One-Time Password

PAN	Permanent Account Number — India income tax identifier
ReKYC	Re-Know Your Customer — periodic re-verification of KYC
REST	Representational State Transfer — API design pattern
SFDC	Salesforce — CRM platform used for case management
SSO	Single Sign-On
UCIC	Unique Customer Identification Code
TX	Database Transaction
UI	User Interface
RDBMS	Relational Database Management System
JPA	Java Persistence API
ORM	Object Relational Mapping
SMTP	Simple Mail Transfer Protocol

5. Design Tools

The ABC ReKYC system utilizes the following design and management tools:

Tool	Purpose
MySQL Workbench	Database schema design, query execution, and ER diagram visualization
IntelliJ IDEA	Primary IDE for Java/Spring Boot backend development
Git / Bitbucket	Source code version control and collaboration

Jenkins	CI/CD pipeline automation — build, dependency check, deploy
Docker	Containerized deployment of the application
Postman	API testing and documentation
JIRA	Project tracking and user story management
SonarQube	Static code quality and security analysis

6. System Overview

6.1 Business Architecture:

The ABC ReKYC system operates across five functional domains. Two types of actors (Customer and Admin) interact through a Frontend with the backend system. The backend is organized into a Security Layer, Core Application, Schedulers, and Storage Layer. External integrations (CyberArk, Email/SMS, Salesforce, Databricks, PAN Validation, OTP API) are shown as external systems.

6.1.1 Authentication & Security Module

The Authentication module manages secure access for both customers and administrators. For customers, it provides OTP-based authentication via mobile or email — customers request a one-time password, validate it, and receive a secure JWT token. For administrators, it implements CyberArk OIDC Authorization Code Flow — admins authenticate via the corporate identity provider, and the system exchanges the authorization code for user identity claims to issue an Admin JWT.

6.1.2 Customer Portal Module

The Customer Portal module provides authenticated customers with the ability to view and update their KYC information. Customers can update personal details (name, address, contact information), PAN card details with real-time PAN validation, beneficiary (nominee) information (add, modify, delete), and upload supporting identity and address documents. All submissions go through a 3-Phase Update

Architecture to prevent database connection pool exhaustion during Salesforce API calls. Customer changes are tracked field-by-field with full audit history. Every submission generates a unique KYC Reference ID.

6.1.3 Reminder Engine Module

The Reminder Engine automates the multi-stage notification campaign for customers who have an upcoming or overdue ReKYC date. The engine runs daily at 2:00 AM and processes six reminder stages:

Stage	Timing	Action	Document Upload
INITIAL	1 month before due date	Email + SMS with portal link	No
REMINDER_1	27 days before due date	Email + SMS follow-up	No
REMINDER_2	15 days before due date	Email + SMS urgent reminder	Yes — PDF to S3
REMINDER_3	1 day after due date	Email + SMS overdue notice	No
REMINDER_4	4 days after due date	Email + SMS escalation	No
REMINDER_5	15 days after due date	Email + SMS final notice	Yes — PDF to S3

Each notification includes a secure, time-limited TinyURL pointing to the customer ReKYC portal. Customers who have already completed their ReKYC are automatically excluded from further reminders.

Strategy Pattern for Dual-Mode Reminders:

The reminder engine uses a Strategy Pattern (`RekycLevelStrategy` interface) to support both UCIC and LAN modes:

Component	Purpose
`RekycLevelStrategy` interface	Defines `findDueForReminder()` and `createTracker()`
`SingleLanRekycReminderImpl`	UCIC mode — one Re-KYC per person covering all LANs

Per-LAN implementation	LAN mode — individual Re-KYC per loan account
`RekycReminderRouter`	Selects the active strategy based on `app.rekyc.level` config

UCIC Mode Tracker:

- `tracker.loan\_account\_id` = NULL
- `tracker.tracking\_level` = UCIC
- `tracker.source\_lan\_no` = LAN with latest `last\_rekyc\_date` (for risk category lookup)

LAN Mode Tracker:

- `tracker.loan\_account\_id` = FK to specific loan
- `tracker.tracking\_level` = LAN
- `tracker.source\_lan\_no` = NULL

6.1.4 SFDC Case Management Module

The SFDC module manages the integration with Salesforce for cases where customers submit changes requiring salesforce review. When a customer submits updated KYC information with changes, the system automatically raises a Salesforce case with the changes and attached documents. The SFDC Query Scheduler runs daily at 10:00 PM to query Salesforce for case status updates. When a case is resolved by the salesforce, the scheduler updates the customer ReKYC dates in the local database.

6.1.5 Admin Dashboard Module

The Admin Dashboard module provides admins and compliance officers with real-time visibility into the ReKYC process. It displays:

- ◆ KYC completion metrics (total due, completed, pending, clicked but not submitted)
- ◆ Risk distribution by category (High / Medium / Low)
- ◆ Monthly completion trends (6-month chart)

- ◆ Salesforce case status distribution
- ◆ Per-stage reminder statistics (targeted, sent, completed, remaining)
- ◆ Individual customer profiles with full change history and case status

Dual-Mode Admin List:

Mode	List Shows	Row Represents	Status Derived From
LAN	Per-loan rows	Each `loan_account`	Tracker by `loan_account_id` + SFDC case
UCIC	Per-person rows	Each `customer_ucic`	`rekyc_reference_id` on UCIC + SFDC case

6.1.6 DWH Synchronization Module

The DWH Sync module maintains bidirectional data synchronization with the Databricks Data Warehouse, which serves as the database source of truth for customer master data.

Forward Sync (1:00 AM daily): Pulls updated customer records from the DWH and upserts them into the local MySQL database.

Reverse Sync: After customers update their KYC information, changes are pushed back to the DWH. The system uses cursor-based pagination to efficiently identify and sync modified records

6.1.7 Audit & History Module

The Audit module maintains a comprehensive and immutable audit trail of all system activities:

- ◆ Field-level change history for every customer update (old value to new value)
- ◆ Beneficiary change history (additions, modifications, deletions)
- ◆ Reminder delivery tracking per customer per stage
- ◆ Salesforce case lifecycle tracking
- ◆ Scheduler execution logs (start time, completion status, records processed, errors)

6.2 Application Architecture

6.2.1 Web Client (React)

The ABC Re-KYC Web Client application is developed to support all major modern web browsers using React as the primary technology stack. React is a JavaScript library used for building efficient and scalable single-page applications (SPA) by creating reusable UI components.

The application uses TypeScript to ensure type safety and maintain high code quality. Vite is used as the build tool to enable fast development and optimized production builds. The UI layer is implemented using Tailwind CSS, which provides a responsive and consistent design system across the application.

The system follows a component-based architecture, allowing reusable UI components, modular development, and improved maintainability.

6.2.1.1 Library Versions

Technology	Version
React	19.x
TypeScript	5.7.x
Vite	7.x
Tailwind CSS	4.x
TanStack Router	1.x
TanStack React Table	8.x
Axios	1.x
React Hook Form	7.x
Zod	4.x
Chart.js	4.x
Node.js	20.x

The following best practices followed in DTS system

- ◆ Implemented a component-based architecture to improve maintainability and scalability.
- ◆ Enabled TypeScript strict typing to enhance code quality and reduce runtime errors.
- ◆ Used React Hook Form with Zod validation for efficient form handling and validation.
- ◆ Managed API communication using Axios with centralized request handling.
- ◆ Implemented TanStack Router for type-safe and structured application routing.
- ◆ Used TanStack React Table to implement advanced table functionalities such as sorting, filtering, and pagination.
- ◆ Developed reusable UI components to ensure consistency across the application.
- ◆ Implemented centralized error handling and API response management.
- ◆ Used Tailwind CSS utility classes for responsive UI development.
- ◆ Implemented role-based and route-based access control for secure navigation and module access.

6.2.1.2 Key Packages

Package	Purpose
react	Core library for building UI components
react-dom	Rendering React components in the DOM
axios	HTTP client for API communication
tailwindcss	Utility-first CSS framework for UI styling
@tanstack/react-router	Type-safe routing for React applications
@tanstack/react-table	Advanced data table functionalities
react-hook-form	Efficient form handling
@hookform/resolvers	Integration between React Hook Form and validation libraries
zod	Schema-based validation library
chart.js	Data visualization library

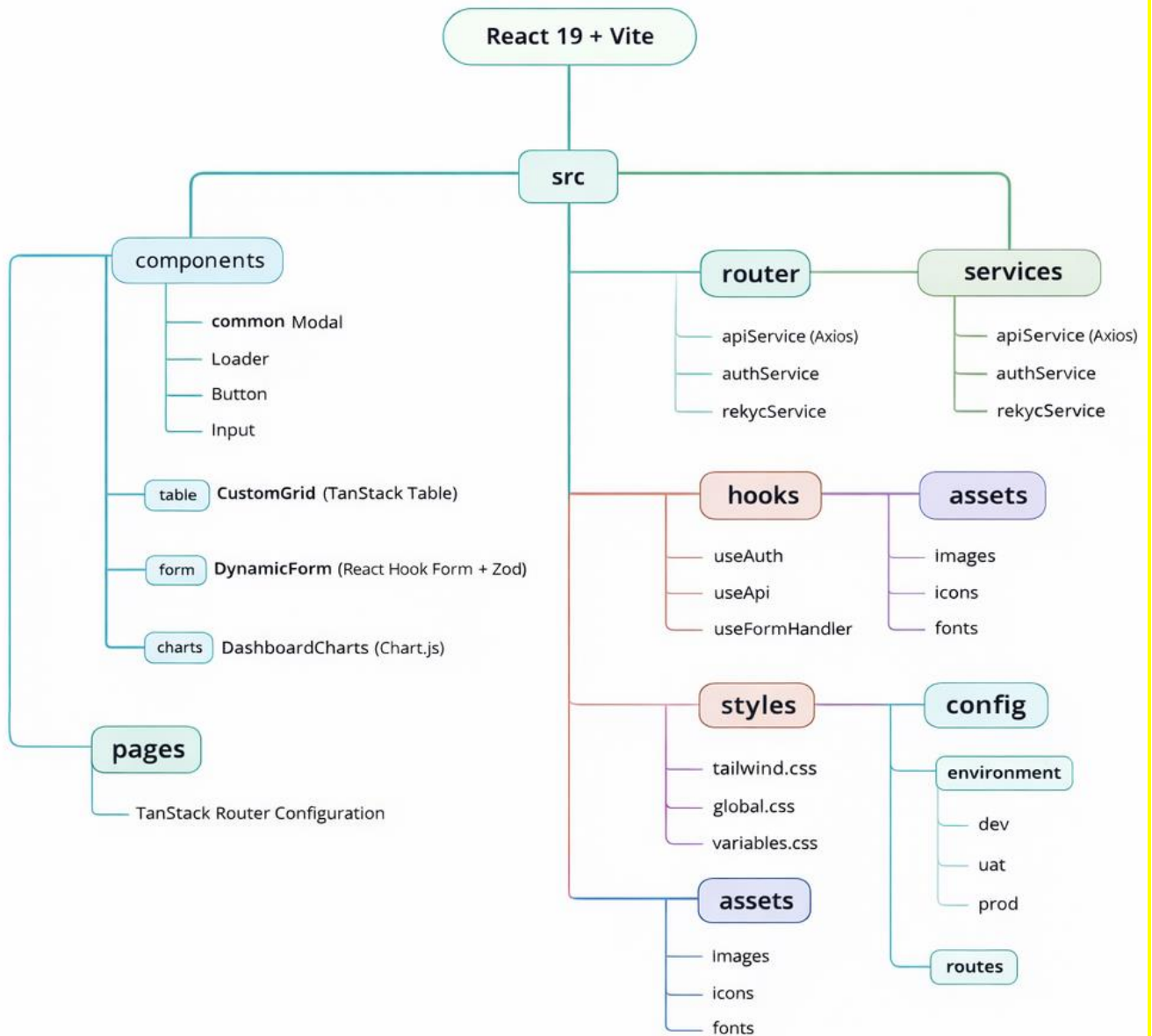
react-chartjs-2	React wrapper for Chart.js
lucide-react	Icon library used across the application
react-hot-toast	Notification and alert system
web-vitals	Performance monitoring

6.2.1.3 Key Re-usable Components

Component	Purpose
Custom Grid	A reusable data grid/table component designed to display tabular data. Supports sorting, filtering, pagination, and column configuration. Built using TanStack React Table. Supports actions such as view, edit, and delete records. Integrates with APIs to dynamically fetch and display data.
Dynamic Form	A flexible form component built using React Hook Form. Supports multiple input types such as text inputs, dropdowns, date pickers, and checkboxes. Includes schema validation using Zod. Supports conditional fields and dynamic field rendering. Can handle both single-section and multi-section forms.
Bulk Upload	Allows users to upload multiple records or files simultaneously. Supports file validation, preview, and preprocessing before submission. Includes error handling and reporting for invalid records.
Export Table Data	Enables users to export table or grid data into CSV format. Supports exporting filtered or selected records. Allows formatting of exported data for improved readability.

Comments	A reusable component used to capture user comments or notes related to specific records. Supports timestamps and user identification. Can be associated with specific forms, workflows, or transactions.
Modal	A reusable dialog/modal component used for displaying forms, confirmations, and detailed information. Supports multiple actions such as submit, cancel, and confirm operations. Designed to be reused across multiple modules of the application.

ABC ReKYC Web Client – Low Level Design (LLD) - Component & Service Structure



6.2.2 Backend Application (Spring Boot)

The ReKYC backend is a Spring Boot 3.3.x monolithic application structured in a layered architecture (Controller to Service to Repository to Database). It exposes a RESTful API consumed by the customer portal frontend and admin dashboard frontend.

6.2.2.1 Technology Stack

Technology	Version	Purpose
Java	21	Primary programming language
Spring Boot	3.3.x	Application framework
Spring Security	6.3.5	Authentication and authorization
Spring Data JPA	3.3.x	ORM and database interaction
Spring Scheduling	3.3.x	Cron-based scheduled tasks
Spring Async	3.3.x	Asynchronous email and SMS sending
Spring Cache	3.3.x	Dashboard response caching
Hibernate	6.x	JPA implementation / ORM
MySQL	8.x	Primary relational database
AWS S3 SDK	Latest	Document storage integration
RestTemplate	Spring 6.x	HTTP client for external APIs
Lombok	Latest	Boilerplate reduction
Maven	3.x	Build tool

6.2.2.2 Best Practices Followed

- ◆ Layered architecture: Controller to Service Interface to Service Impl to Repository
- ◆ @Transactional boundaries carefully managed — 3-phase architecture for SFDC calls
- ◆ @Async for all non-blocking operations (email, SMS)
- ◆ @Cacheable for dashboard summary to reduce DB load
- ◆ Interface-based service design for testability and mocking
- ◆ Custom exception hierarchy with global exception handler
- ◆ Batch DB queries to avoid N+1 query problems in list APIs
- ◆ Cursor-based pagination for large dataset processing (DWH sync)
- ◆ Idempotency in reminder processing via unique DB constraint per customer + stage + dueDate

6.2.2.3 Key Packages

Package	Purpose
com.abc.rekyc.auth	JWT filters, CyberArk OIDC, OTP validation, rate limiting
com.abc.rekyc.customer	Customer entity, services, controllers, schedulers, mappers
com.abc.rekyc.beneficiary	Beneficiary entity, services, controllers
com.abc.rekyc.admindashboard	Dashboard, reports, admin customer views, filter service
com.abc.rekyc.sfdc	Salesforce integration, case entity, SFDC query scheduler
com.abc.rekyc.dwh	DWH sync services, schedulers, RekycMaster entity
com.abc.rekyc.otp	OTP send/validate service and rate limit service

com.abc.rekyc.notification	Email and SMS service interfaces and implementations
com.abc.rekyc.storage	AWS S3 and SharedDrive document storage
com.abc.rekyc.pan	PAN validation service
com.abc.rekyc.audit	Scheduler run logging service
com.abc.rekyc.exception	Custom exception classes and global exception handler
com.abc.rekyc.config	Security, CORS, async, cache, datasource configuration

6.2.2.4 API Endpoints Summary

Module	Base Path	Method(s)	Access
OTP — Send OTP	/api/otp/send	POST	Public
OTP — Validate OTP	/api/otp/validate	POST	Public
Auth — Admin Login URL	/api/auth/admin/login	GET	Public
Auth — Admin Callback	/api/auth/admin/callback	POST	Public
Auth — Current User	/api/auth/me	GET	JWT Required
Customer — Get	/api/customers	GET	Customer JWT

Customer Update	—	/api/customers/update	PATCH	Customer JWT
Customer Download KYC Info	—	/api/customers/download-info	GET	Customer JWT
Customer Download Ref Info	—	/api/customers/kyc-completion-info	GET	Customer JWT
Beneficiary By Customer	—	/api/beneficiaries/customer/{customerNo}	GET	Customer JWT
Beneficiary By ID	—	/api/beneficiaries/{beneficiaryId}	GET	Customer JWT
Dashboard Summary	—	/api/dashboard/summary	GET	JWT Required
Dashboard Reminder Stats	—	/api/dashboard/reminder-stats	GET	JWT Required
Admin Customer List	—	/api/admin/customers/list	GET	JWT Required
Admin Customer Detail	—	/api/admin/customers/{identifier}/details identifier->lan number and ucic number for ucic level rekyc	GET	JWT Required
Admin Customer Changes	—	/api/admin/customers/changes/{identifier}	GET	JWT Required

Admin — Recent Activity	/api/admin/recent?type=exceptions completed	GET	JWT Required
Reports — Download	/api/reports/download	GET	JWT Required
SFDC — Query Case	/api/sfdc/query-case/{caseNumber}	GET	Token

6.2.3 Security Architecture

6.2.3.1 Customer Authentication (OTP to JWT)

Customer requests OTP via POST /api/otp/send (mobile or email). External OTP API delivers the OTP. Customer validates OTP via POST /api/otp/validate. JwtService issues a signed Customer JWT. All subsequent API calls are made with the Bearer JWT. JwtAuthFilter validates the token on every request.

6.2.3.2 Admin Authentication (CyberArk OIDC)

Admin calls GET /api/auth/admin/login and receives the CyberArk OIDC Authorization URL. Browser redirects to CyberArk IDP where admin authenticates with corporate credentials. CyberArk redirects back with an authorization code. Admin calls POST /api/auth/admin/callback with the code. Backend exchanges the code for CyberArk tokens, fetches UserInfo (name, email, sub), creates or updates AdminUser record in DB, and issues an Admin JWT with ROLE_ADMIN claim.

6.2.3.3 JWT Filter Chain

Every incoming request passes through two filters in sequence:

Filter	Order	Handles
AdminJwtFilter	1st	Validates Admin JWT — sets ROLE_ADMIN in SecurityContext
JwtAuthFilter	2nd	Validates Customer JWT — sets customer identity (skips if admin already set)
JwtAuthEntryPoint	—	Returns HTTP 401 for unauthenticated requests

6.2.4 Three-Phase Customer Update Architecture

A critical architectural decision is the 3-Phase Transaction Design for customer updates. This prevents HikariCP database connection pool exhaustion during long Salesforce API calls .

Phase	Transaction	Action
PRE-PHASE	Read-only TX	Load customer, map request, generate rekycReferenceId. DB connection released after commit.
PHASE 1	Write TX	Count all changes. If changes > 0: create RekycCase (status=PENDING). DB connection released.
PHASE 2	NO TRANSACTION	Call Salesforce CreateCase API + attach documents. If SFDC fails: RekycCase=FAILED, Phase 3 never runs.
PHASE 3	Write TX	Save CustomerHistory + BeneficiaryHistory. Mark tracker COMPLETED. Update customer. Save.
POST-PHASE	Async no TX	Send KYC completion email to customer asynchronously.

Phase 3 Safety Net: If Phase 3 fails after SFDC success, the system retries once automatically. If the retry also fails, the API returns success to the customer to prevent duplicate Salesforce case creation. A CRITICAL log alert is raised for manual reconciliation.

6.2.5 Scheduler Architecture

The system runs five background schedulers, all using Spring @Scheduled with Asia/Kolkata timezone. Each scheduler logs its execution to the scheduler_run_log table.

#	Scheduler	Class	Cron / Frequency	Purpose
1	ReKYC Reminder	RekycReminderScheduler	Daily 2:00 AM	Process all 6 reminder stages, send Email + SMS notifications
2	DWH Sync	DwhSyncScheduler	Daily 1:00 AM	Sync customer data from Databricks Data Warehouse to MySQL
3	SFDC Case Query	SfdcCaseQueryScheduler	Daily 10:00 PM	Query Salesforce for case resolution status, update customer dates

Reminder Scheduler — Reminder Stage Details:

#	Stage	Trigger Criteria	Action	Document Generated
---	-------	------------------	--------	--------------------

1	INITIAL	nextRekycDate = today + 1 month	Email + SMS	No
2	REMINDER_1	nextRekycDate = today + 27 days	Email + SMS	No
3	REMINDER_2	nextRekycDate = today + 15 days	Email + SMS	Yes — PDF uploaded to S3
4	REMINDER_3	nextRekycDate = today – 1 day	Email + SMS	No
5	REMINDER_4	nextRekycDate = today – 4 days	Email + SMS	No
6	REMINDER_5	nextRekycDate = today – 15 days	Email + SMS	Yes — PDF uploaded to S3

6.2.6 Database Layer (MySQL)

The MySQL database serves as the primary persistence store. Key tables:

Table	Purpose
customer_ucics	Person-level personal data — name, contact, address, PAN, DMS folder. In UCIC mode: also stores `rekyc_reference_id`, `last_rekyc_date`, `next_rekyc_date`
customers	System-level linking — `customer_no`, `system`, `lob`, `ccc`, `applicant_type`. FK to `customer_ucics`

loan_accounts	Per-loan data — `lan_no`, `risk_category`. In LAN mode: also stores `rekyc_reference_id`, `last_rekyc_date`, `next_rekyc_date`. FK to both `customers` and `customer_ucics`
beneficiaries	Customer nominee / beneficiary records
customer_history	Field-level audit trail for all customer changes
beneficiary_history	Audit trail for beneficiary additions, changes, deletions
rekyc_reminder_tracker	Per-stage reminder tracking with statuses (OPEN / COMPLETED)
rekyc_cases	Salesforce case tracking — case number, status, document status
admin_user_details	Admin user accounts sourced from CyberArk
scheduler_run_log	Execution log for all schedulers (start, end, status, counts, errors)
dwh_sync_metadata	DWH sync job state and last successful sync timestamp
rekyc_master	DWH mirror — Databricks source of truth for customer master data

Risk-Based ReKYC Interval (from Customer.updateNextRekycDate()):

Risk Category	ReKYC Frequency	nextRekycDate Calculation
High	Every 2 years (24 months)	lastRekycDate + 24 months
Medium	Every 8 years (96 months)	lastRekycDate + 96 months
Low	Every 10 years (120 months)	lastRekycDate + 120 months

6.2.7 Document Storage (AWS S3)

AWS S3 is used for storing reminder PDF documents generated at REMINDER_2 and REMINDER_5 stages. Document IDs (S3 keys) are stored in rekyc_reminder_tracker.document_id. An alternate SharedDrive implementation is available via configuration. Customer-uploaded documents (Base64 encoded) are passed directly to Salesforce as case attachments and are not persisted to S3.

6.2.8 External System Integrations

System	Purpose	Integration Type
CyberArk IDP	Admin SSO — OIDC Authorization Code Flow	REST API (OAuth 2.0 / OIDC)
Email Service	OTP delivery, reminders, completion emails, daily summary	REST API (async)
SMS Service	OTP delivery, reminder SMS notifications	REST API (async)
Salesforce (SFDC)	Case creation, document attachment, case status query	REST / SOAP API
Databricks DWH	Customer master data source and reverse sync target	JDBC / REST API
PAN Validation API	Real-time PAN card validation	REST API
OTP API	Send and validate OTPs (mobile and email)	REST API
TinyURL API	Generate short expiring URLs for reminder notification links	REST API

External api Endpoints:

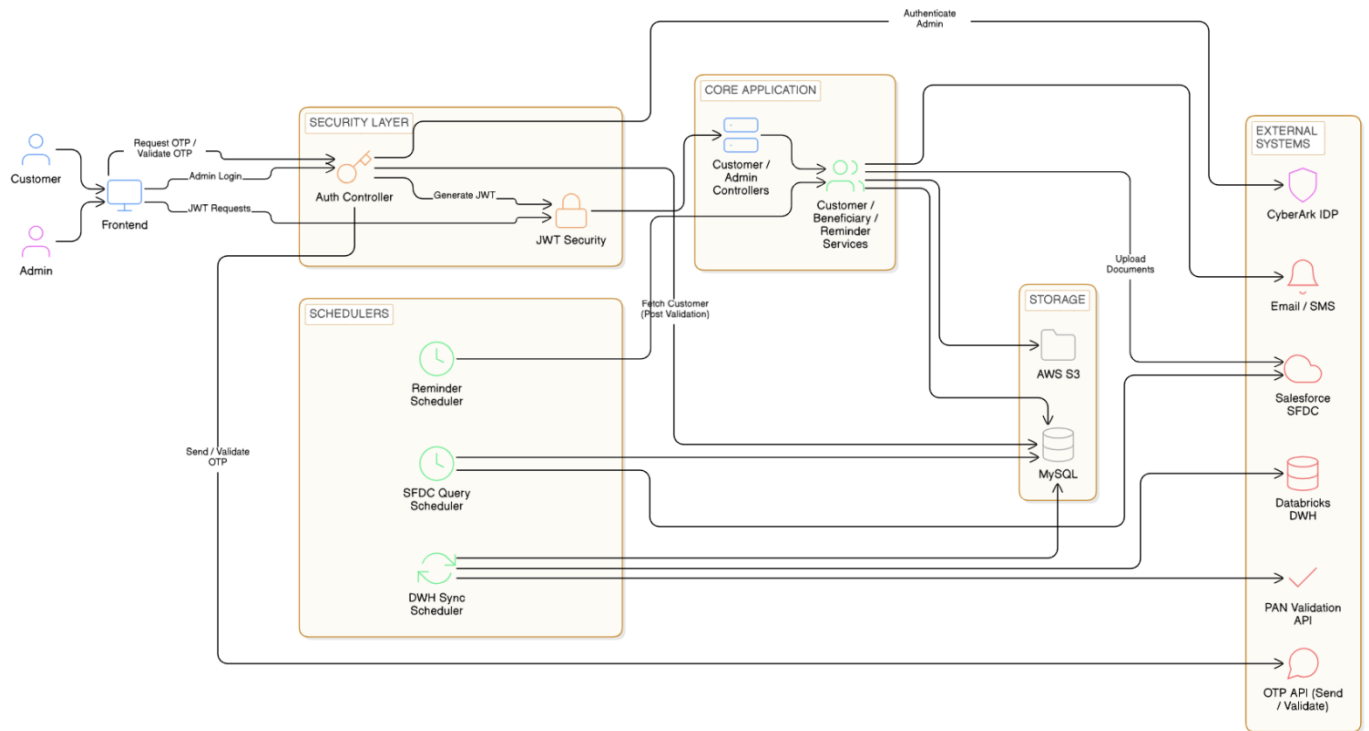
- 1./abccs/cognito/token
- 2./crm-abfl/rekyc-pan-validate
- 3./sfdc-abfl/rekyc-createcase

- 4./sfdc-abfl/rekyc-querycase
- 5./sfdc-abfl/rekyc-updatecase
- 6./tinyurl/getshortURL
- 7./abccs/getotp
- 8./abccs/validateotp
- 9./crm-abfl/sinch_send-email
- 10./abccs/sendsms

6.2.9 In-App Notification Design

All customer and admin notifications are handled asynchronously using Spring @Async. This ensures API response times are not affected by notification delivery latency.

Notification	Trigger	Channel
OTP delivery	Customer requests OTP	Email or SMS
ReKYC reminder	2AM daily scheduler	Email + SMS
ReKYC completion confirmation	Customer submits ReKYC	Email
Daily scheduler summary	After each scheduler run	Email (to admin recipient)



6.3 Deployment Architecture

6.3.1 CI/CD Pipeline with Jenkins

Backend CI/CD Pipeline:

Step	Action	Tool
1	Code Commit — Developer pushes to Bitbucket (abc_rekyc_backend)	Git / Bitbucket
2	Build — Jenkins triggers Maven build; compiles and packages the Spring Boot fat JAR	Jenkins + Maven

3	Code Quality — SonarQube static analysis for code quality and security hotspots	SonarQube
4	Docker Image Build — Jenkins builds a Docker image from the packaged JAR	Docker
5	Push to Registry — Docker image pushed to Docker Hub (impiger/abc_rekyc_backend)	Docker Hub
6	Deploy — Backend container deployed to target server via SSH + docker-compose	Docker Compose + SSH

Frontend CI/CD Pipeline:

Step	Action	Tool
1	Code Commit — Developer pushes to Bitbucket (abc_rekyc_frontend)	Git / Bitbucket
2	Build — npm ci install, then vite build (outputs production assets to /dist)	Node.js 22 + Vite
3	Vulnerability Scan — Gype container image vulnerability scanning	Gype
4	Code Quality — SonarQube analysis (project key: abc_rekyc_frontend)	SonarQube

5	Docker Image Build — Multi-stage build: Node 22 (build) → Nginx (serve)	Docker
6	Push to Registry — Docker image pushed to Docker Hub (impiger/abc_rekyc_frontend)	Docker Hub
7	Deploy — Frontend container deployed to server via docker-compose (service: frontend)	Docker Compose + SSH

6.3.2 Traffic Flow & Load Balancing

The system uses Nginx as a reverse proxy handling SSL termination and domain routing. After SSL termination, Nginx forwards requests to the Spring Boot application container. This ensures secure and efficient traffic management for the application.

6.3.3 Application Container

The application is deployed as a Docker container running the Spring Boot JAR. Each environment (Dev, UAT, Production) runs an isolated container instance. Environment-specific configuration is injected via environment variables — no hardcoded secrets.

6.3.4 Database & Storage Layer

Component	Technology	Purpose
Primary Database	MySQL 8.x	All transactional data — customers, cases, history, trackers
Document Storage	AWS S3	Reminder PDF documents
In-Memory Cache	Spring Cache	Dashboard summary caching (in-process)

6.3.5 Monitoring & Logging

Area	Approach
Application Logs	SLF4J + Logback — structured logging with log levels
Scheduler Execution	scheduler_run_log table — DB-based execution audit trail
Error Alerts	ERROR / CRITICAL level log messages with full stack trace
API Performance	Response time logging in service layer

6.3.6 Key Environment Configuration

Environment Variable	Purpose
APP_BASE_URL	Application Api base URL
SERVER_PORT	Tomcat server port
CORS_ALLOWED_ORIGINS	CORS allowed frontend origins
DB_HOST, DB_PORT, DB_NAME	MySQL database connection
DB_USERNAME, DB_PASSWORD	MySQL credentials
EXTERNAL_API_BASE_URL	Base URL for all external REST APIs
EXTERNAL_API_TIMEOUT	HTTP call timeout in seconds
JWT_SECRET	HMAC SHA key for JWT signing
SMS_SOURCE	SMS source identifier
JWT_EXPIRATION_MS	JWT token lifetime in seconds
CYBERARK_APPLICATION_ID	CyberArk application ID
CYBERARK_TENANT_URL	CyberArk OIDC tenant URL
CYBERARK_CLIENT_ID	CyberArk OAuth2 client ID
CYBERARK_CLIENT_SECRET	CyberArk OAuth2 client secret

AWS_S3_ACCESS_KEY	AWS S3 access key for document storage
AWS_S3_SECRET_KEY	AWS S3 secret key for document storage
AWS_S3_REGION	AWS S3 region
AWS_S3_BUCKET_NAME	AWS S3 bucket name
REMINDER_DOCUMENT_STORAGE_S3	true = use S3, false = use Shared Drive
STORAGE_SHARED_DRIVE_BASE_PATH	Shared drive base path (when S3 disabled)
DMS_PARENT_FOLDER_INDEX	DMS parent folder index for document uploads
SCHEDULER_SUMMARY_EMAIL	Email recipient for daily scheduler summary
REKYC_BASE_URL	Customer portal URL (used in reminder links)

7. Goals and Constraints

7.1 Usability

- ◆ Customer portal accessible via standard web browsers (Chrome, Firefox, Safari, Edge)
- ◆ OTP login must complete within 60 seconds of request
- ◆ API responses for customer-facing endpoints must return within 3 seconds under normal load
- ◆ Admin dashboard must load summary metrics within 5 seconds (supported by @Cacheable)
- ◆ All notifications (email, SMS) sent asynchronously — no impact on API response time
- ◆ Error messages must be meaningful and actionable for both customers and administrators

7.2 Security

- ◆ All API communication over HTTPS (TLS 1.2+)
- ◆ Customer authentication via OTP — rate-limited to max 3 requests per 15 minutes
- ◆ Admin authentication via CyberArk OIDC — no admin password stored in the system
- ◆ JWT tokens signed with a secure secret key; validated on every request by dual-filter chain
- ◆ Beneficiary deletion includes ownership verification to prevent cross-customer unauthorized access

- ◆ All sensitive configuration stored as environment variables — never hardcoded in source code
- ◆ SonarQube code scanning for security hotspots (SQL injection, XSS, etc.).

7.3 Platform dependent

- ◆ Backend: Java 21 — compatible with any JVM-capable OS (Linux preferred for production)
- ◆ Database: MySQL 8.x — compatible with any MySQL-compliant deployment environment
- ◆ Storage: AWS S3 — alternative SharedDrive implementation available for non-cloud environments
- ◆ Deployment: Docker-based — platform-independent containerized runtime
- ◆ Browser: Standard modern browsers supported (Chrome, Firefox, Safari, Edge — no IE support).

7.4 Performance

7.4.1 Transactions

- ◆ Database transactions kept short and focused — targeting less than 100ms per transaction
- ◆ 3-phase update architecture ensures no DB connection held during Salesforce API calls (up to 99 seconds)
- ◆ HikariCP connection pool configured to prevent exhaustion under peak load
- ◆ `@Transactional(readOnly = true)` used for all read operations to optimize DB resource usage.

7.4.2 Concurrency Support

- ◆ Spring Boot handles concurrent requests via embedded Tomcat thread pool
- ◆ Async email/SMS operations run in a dedicated thread pool configured via `AsyncConfig`
- ◆ Scheduled jobs run in isolation — each scheduler has its own execution context
- ◆ Batch DB queries prevent N+1 query problems in list and dashboard APIs
- ◆ `@Cacheable` on dashboard summary reduces DB load as admin usage grows

7.5 Maintainability

7.5.1 Components and Objects

- ◆ Layered architecture (Controller → Service → Repository) ensures clear separation of concerns
- ◆ Interface-based service design allows mocking and unit testing without infrastructure dependencies
- ◆ Custom exception hierarchy with a centralized GlobalExceptionHandler for consistent error responses
- ◆ BaseTimestampEntity and BaseHistory as shared superclasses reduce duplication across entities
- ◆ CustomerFilterService shared by both Admin Customer List and Reports modules to avoid code duplication
- ◆ StorageConfig factory pattern allows switching between S3 and SharedDrive without code changes

7.5.2 Third Party Vendors

Vendor	Integration	Fallback / Error Handling
Salesforce (SFDC)	Case management API	RekycCase marked FAILED; customer data unchanged (Phase 2 abort)
CyberArk	Admin OIDC authentication	None — required for admin access
Email Service	Notification delivery	Logged; async — does not block main flow
SMS Service	Notification delivery	Logged; async — does not block main flow
TinyURL API	Short URL generation	Tracker marked FAILED; reminder skipped for customer this run
AWS S3	Document storage	SharedDrive implementation available via config switch

Databricks DWH	Customer master data	Local MySQL data used until next successful sync
OTP API	OTP send and validate	Error returned to customer frontend
PAN Validation	PAN authenticity check	Validation error returned to customer

7.6 Scalability

- ◆ Docker-based deployment supports horizontal scaling by running multiple container instances behind a load balancer
- ◆ Stateless API design (JWT authentication) — any instance can serve any request
- ◆ Batch DB queries scale efficiently as customer volumes grow
- ◆ @Cacheable on dashboard summary reduces DB load as admin usage grows
- ◆ AWS S3 scales automatically for document storage regardless of volume
- ◆ Scheduler jobs can be offloaded to a dedicated scheduler instance if needed

7.7 Reliability

- ◆ Phase 3 retry logic ensures customer updates survive transient DB failures after SFDC success
- ◆ FailureRetryScheduler ensures failed notifications and document uploads are eventually delivered
- ◆ Idempotent reminder processing — unique DB constraint per customer + stage + due date prevents duplicate notifications
- ◆ scheduler_run_log provides complete execution history for audit and debugging
- ◆ All external API calls have structured error handling with specific exception types (SfdcException, ExternalApiException, etc.)
- ◆ Zero-change submissions bypass SFDC entirely — no risk of SFDC failures for no-change customers

7.8 Compatibility

7.8.1 Database

Database	Version	Usage
MySQL	8.x	Primary transactional database — all application data

All ORM operations use JPA/Hibernate, ensuring SQL dialect compatibility. MySQL-specific features are minimized to maintain portability.

7.8.2 Technology

Technology	Version
Java	21
Spring Boot	3.3.x
Spring Security	6.3.5
Spring Data JPA	3.3.x
Hibernate	6.x
Maven	3.x
Docker	Latest stable
MySQL Connector	8.x
AWS SDK	Latest

8. Design

8.1 The User Interface Environment

The ABC ReKYC backend exposes a RESTful JSON API consumed by two frontend applications:

- ◆ Customer Portal (Web): A browser-based single-page application where customers log in via OTP, review their KYC information, make updates, upload documents, and submit their ReKYC.
- ◆ Admin Dashboard (Web): A browser-based single-page application where administrators log in via CyberArk SSO to monitor compliance metrics, view per-customer status, access change histories, and generate reports.
- ◆ Both frontends communicate exclusively with the backend via REST APIs. The backend enforces all business rules, validations, and security. The frontend is responsible only for presentation.

8.2 Module details

8.2.1 Master Data

Master Data	Source	Table	Managed By
Customer Profiles	Databricks DWH (daily sync)	customers	DWH Sync Scheduler
Beneficiary Records	Customer submissions	beneficiaries	Customer Portal
Admin Users	CyberArk IDP (on login)	admin_user_details	Auth Module
Risk Categories	DWH / Customer record	customers.risk_category	DWH Sync Scheduler
ReKYC Due Dates	Calculated from risk category	customers.next_rekyc_date	KycCompletionService
Salesforce Cases	SFDC API	rekyc_cases	SFDC Module
Reminder Trackers	Reminder Scheduler	rekyc_reminder_tracker	Reminder Engine
Scheduler Logs	System generated	scheduler_run_log	All Schedulers

8.2.2 Database

Entity Relationship Summary:

- ♦ customers is the central entity, linked to customer_history, beneficiaries, rekyc_reminder_tracker, and rekyc_cases
- ♦ beneficiaries is linked to beneficiary_history for change tracking
- ♦ customer_history and beneficiary_history are grouped by update_id (rekycReferenceId) for per-submission change views
- ♦ rekyc_reminder_tracker has a unique constraint on (customer_id, reminder_stage) ensuring idempotency
- ♦ rekyc_cases has a unique constraint on rekyc_reference_id — one case per submission
- ♦ admin_user_details, scheduler_run_log, and dwh_sync_metadata are standalone operational tables

Key Database Constraints:

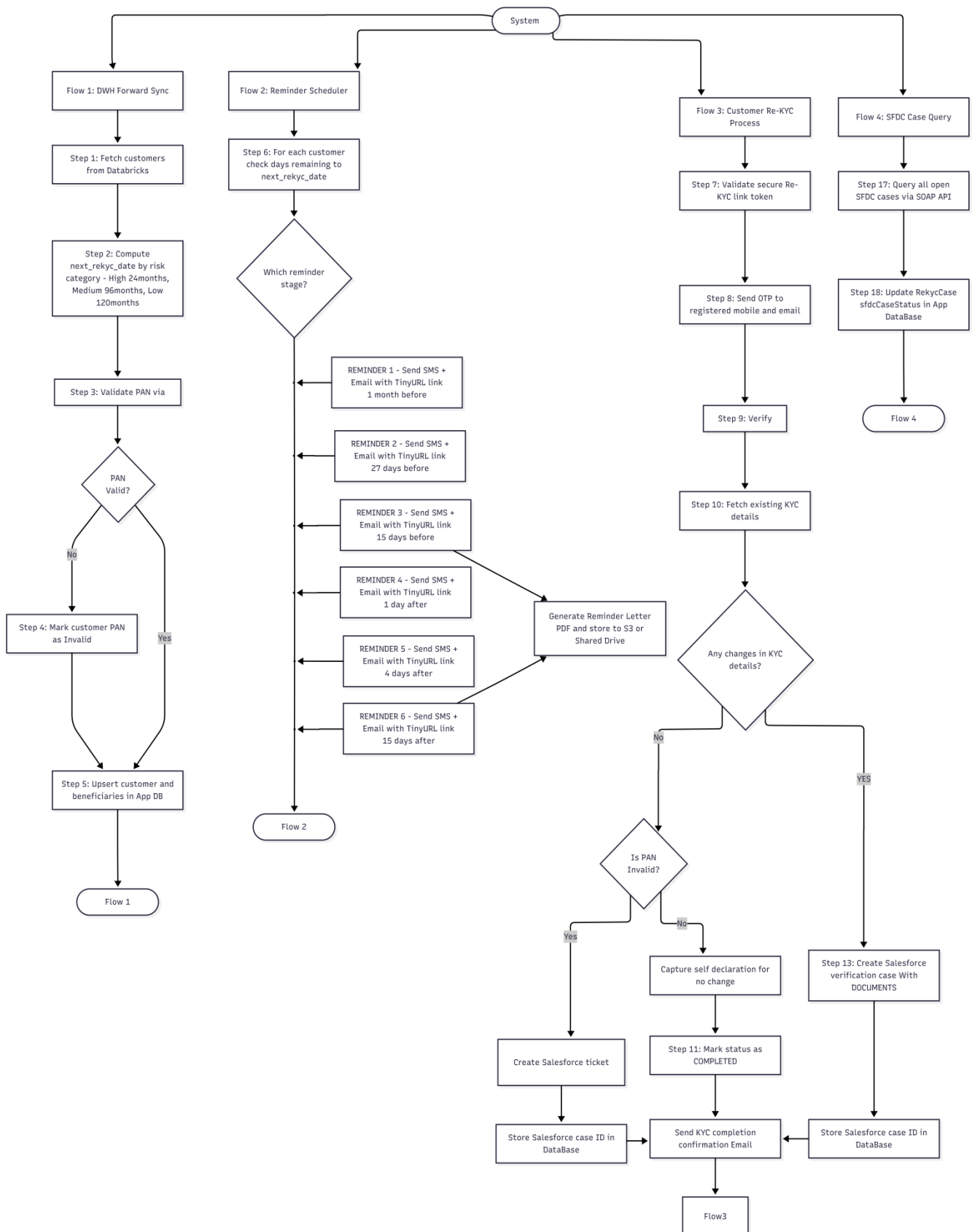
Table	Unique Constraint	Purpose
customers	customer_no, email, mobile1, pan	Prevent duplicate customer records
beneficiaries	beneficiary_id	Unique beneficiary identifier
rekyc_cases	rekyc_reference_id	One case per submission
admin_user_details	sub	CyberArk stable subject identifier

8.2.3 Integration Sequence

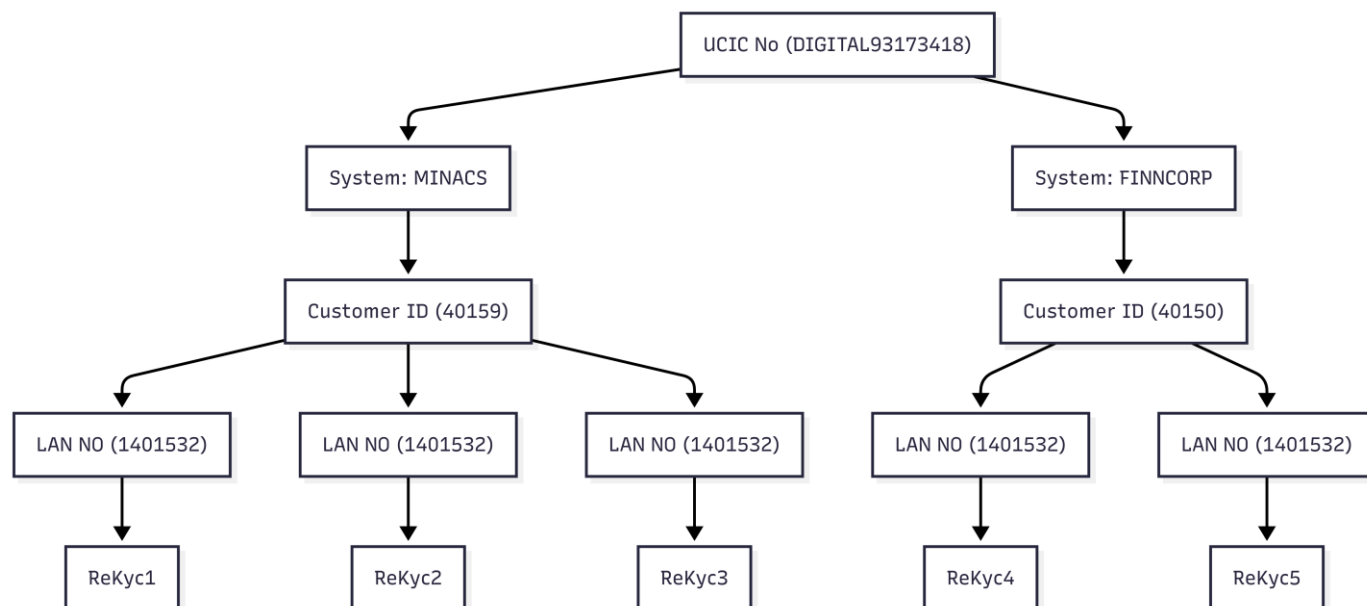
The following is the sequence by which the integration will happen for the primary customer update flow:

Step	Integration	Description
1	OTP API to Email/SMS Service	Customer requests OTP; vendor delivers via email or SMS
2	OTP API	Customer validates OTP; backend issues Customer JWT

3	MySQL READ	Backend loads customer and beneficiaries for portal display
4	MySQL WRITE (Phase 1)	RekycCase created with status=PENDING before SFDC call
5	Salesforce API (Phase 2)	Create case and attach documents; RekycCase updated with sfdcCaseNumber, status=New
6	MySQL WRITE (Phase 3)	CustomerHistory and BeneficiaryHistory saved; customer updated; tracker marked COMPLETED
7	Email Service (async)	KYC completion confirmation email sent to customer
8	Salesforce API (7PM Scheduler)	Nightly poll for case resolution; MySQL updated when Resolved
9	Databricks DWH (Reverse Sync)	Updated customer records pushed back to DWH after submission
10	Databricks DWH (1AM Scheduler)	Fresh customer master data pulled from DWH to MySQL



LAN based rekyc flow:



8.2.4 Reports

The Reports module provides the following data for admin use:

Report	Description
Customer Status Report	All customers with current unified status — filterable by risk, status, search term
Completion Trend Report	Monthly ReKYC completion counts over last 6 months
Reminder Stage Statistics	Per-stage: targeted, sent, completed, remaining counts
SFDC Case Distribution	Monthly Salesforce case status breakdown (New, Resolved, Rejected, etc.)
Risk Distribution Report	Customer count by risk category (High / Medium / Low)
Recent Activity Report	Recent customer submissions and status changes

Admin Unified Status Logic (applied to all customer list and report views):

Tracker Status	SFDC Case Status	Display Status	Meaning
No tracker record	—	No ReKYC Needed	Customer not in active ReKYC cycle
OPEN	—	OPEN	Customer has not yet submitted
COMPLETED	No case	COMPLETED	Submitted with no changes (zero-change submission)
COMPLETED	New	New	Submitted; Salesforce case raised; sfdc not yet reviewed
COMPLETED	In Progress	In Progress	sfdc is reviewing the case
COMPLETED	Resolved	Completed	Both customer submitted and sfdc resolved the case